Introdução à Lógica de Programação

MÓDULO 1 SEMANA 02

@SCTI.SC

AGENDA

- O que é Python?
- Tipos de dados em Python
- Variáveis
- Operadores
- Google Colab

2

O que é Lógica de Programação?

O que é Lógica de Programação?

É a organização sequencial e eficiente de instruções para resolver um problema, definindo entradas, saídas e os passos da solução antes mesmo de codar.

- Organização: Ordem sequencial e coerente de instruções.
- Planejamento: Entender entradas, saídas e processos.
- Resolução: Pensar no problema antes de escrever o código.

Como um Humano Pensa

“Preciso fazer café. Pego a cafeteira, coloco água, coloco o pó, ligo e espero."

Como um Programador Pensa

Define as entradas (água, pó),
Processa Etapas Sequenciais
Verifica condições (tem água?)
Produz a saída (café pronto).

Como o Python Surgiu?

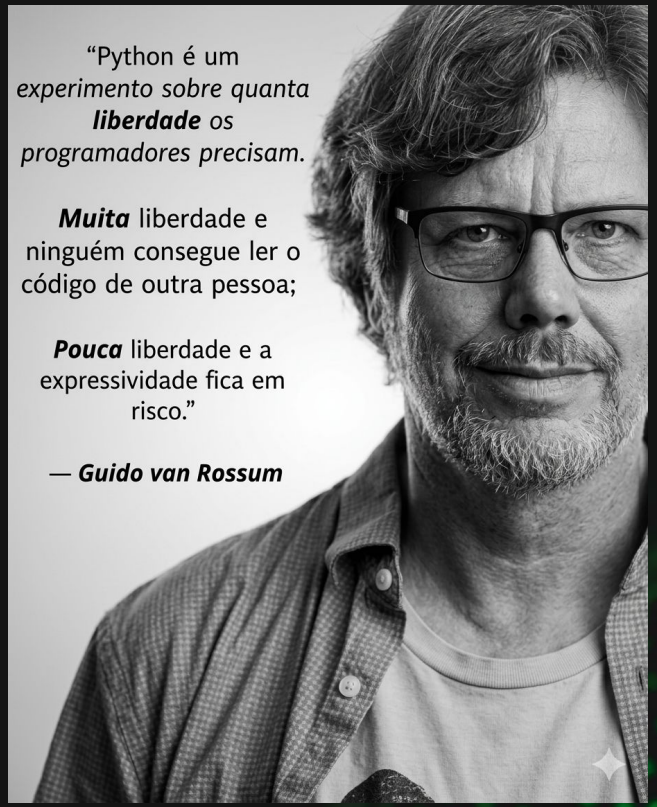
Ano: 1991

Por que foi criada?
Tornar a programação mais simples e produtiva

Principais ideias:
Código fácil de ler e escrever
Sintaxe simples e organizada
Uma forma clara de resolver problemas

Curiosidade:
Nome inspirado no grupo Monty Python

“O Python foi criado para facilitar a vida dos programadores, focando em clareza e simplicidade, diferente de linguagens mais complexas da época.”



Porque o Python é a escolha certa?

Sintaxe Simples

Código próximo ao inglês natural, reduzindo a curva de aprendizado para quem está começando.

Alta Demanda

Usado em Data Science, IA , automação, back-end e APIs. Uma das linguagens mais requisitadas em vagas de tecnologia.

Ecossistema Rico

Milhares de bibliotecas prontas (Pandas, NumPy, Django, Flask) aceleram o desenvolvimento profissional.

Versátil

Da análise de dados à criação de sites e automação de tarefas repetitivas no dia a dia corporativo.

Pensando como um Desenvolvedor

Antes de codificar, todo desenvolvedor experiente passa por um processo mental estruturado. Esse raciocínio é o que diferencia quem apenas "digita código" de quem **resolve problemas com tecnologia**.

01

Entenda o Problema

Leia o enunciado com atenção. Identifique: O que é dado? O que é pedido? Existem restrições?

02

Planeje a Solução

Escreva os passos em português antes de codificar. Esse rascunho chama-se **pseudocódigo** ou **algoritmo**.

03

Codifique e Teste

Transforme o algoritmo em código Python. Teste com diferentes entradas, incluindo casos extremos (zero, negativo, vazio).

04

Refine e Melhore

Revise o código. Está legível? Há repetições desnecessárias? Um bom código conta uma história clara.

Dica de entrevista: Recrutadores avaliam o raciocínio, não apenas o código final. Verbalizar o processo é um diferencial.

Porque os iniciantes amam o Python?

- Sintaxe limpa (menos “pontinhos e parênteses” que outras linguagens).
- Comunidade enorme e muitos tutoriais gratuitos.
- Bibliotecas prontas que aceleram (ex.: para Excel/CSV, web, gráficos, IA).
- Funciona no Windows, macOS, Linux e até no Google Colab (no navegador).

Algumas tarefas que podemos fazer com Python

- Análise e processamento de dados
- Automação de tarefas
- Aplicações Web e APIs
- Sistemas Simples

Tipos de dados em Python

Os tipos de Dados Fundamentais

Compreender os tipos de dados é essencial para evitar erros e escrever código eficiente. Cada tipo tem comportamentos e operações específicas.

int — Inteiro

Números sem casas decimais. Ex: idade = 25 | ano = 2024

float — Decimal

Números com ponto flutuante. Ex: salario = 3500.50 | pi = 3.14

str — Texto

Cadeia de caracteres entre aspas. Ex: nome = "Maria" | cidade = 'São Paulo'

bool — Booleano

Verdadeiro ou Falso. Ex: ativo = True | logado = False

```
# Verificando o tipo de uma variável
nome = "Carlos"
print(type(nome))   # <class 'str'>
idade = 30
print(type(idade))  # <class 'int'>
```

Variáveis em Python

Variáveis

Podemos definir variável como um nome (identificador) que aponta para um objeto na memória, para que assim ele guarde e referencie um valor que poderá ser usado ou modificado posteriormente no programa.

Boas Práticas: Nomes de variáveis

Nomes de variáveis não têm limite de tamanho. Podem conter letras quanto números, mas não podem começar com um número. É possível utilizar letras maiúsculas, embora a convenção recomende utilizar apenas letras minúsculas para nomear variáveis.

❌ Nomes Ruins	✅ Nomes Bons
x = 25 — o que é x?	Idade_usuário = 25
y = 3500.50 — sem contexto	salario_mensal = 3500.50
a1 = "Maria" — confuso	nome_cliente = “Maria”
NOME = "João" — caixa alta é para constantes	TAXA_IMPOSTO = 0.15 (constante)

Palavras Reservadas

Palavras Reservadas			
False	await	else	import
None	break	except	in
True	class	finally	is
and	continue	for	lambda
as	def	from	nonlocal
assert	del	global	not
async	elif	if	or
raise	try	with	pass
return	yield	while	

Python possui palavras reservadas (Keywords) com significados especiais. Elas não podem ser usadas como nomes de variáveis para evitar conflitos.

snake_case

É um padrão de escrita onde as palavras são minúsculas e separadas por underscore _.

Ex:
data_nascimento, total_vendas

Por que devemos usar?

PEP 8 (convenção oficial) recomenda snake_case para variáveis, funções e métodos porque melhora a leitura de nomes longos é mais legível que dataDeUltimaCompra e mantém consistência com a maior parte das bibliotecas Python.

Para Fixar!

1 - Não comece sua variável com números.

2 - Não separe os nomes (ex: quantidade de pessoas).

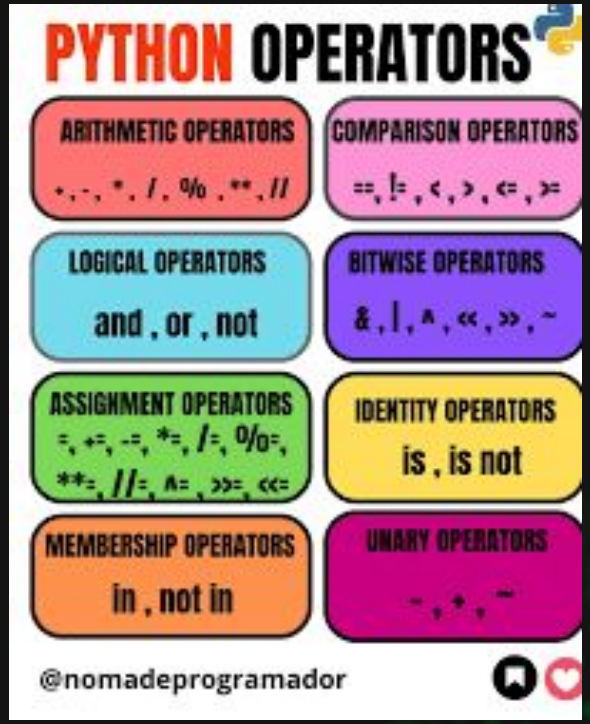
3 - O único caractere especial permitido é o _.

4 - Letras maiúsculas se diferenciam das minúsculas em linguagens CaseSensitive.

Operadores - Conceito

Operadores em Python

Os operadores em Python são símbolos ou palavras-chave especiais que realizam operações em valores e variáveis (chamados operandos). Eles são a base para construir expressões e lógica em qualquer programa.



Operadores – Aritméticos

Os operadores aritméticos são usados para realizar operações matemáticas.

Operador	Descrição	Exemplo
a + b	Adição	4 + 5 = 9
a - b	Subtração	7 - 2 = 5
a * b	Multiplicação	9 * 2 = 18
a ** b	Potência	9 ** 9 = 81
a / b	Divisão real	10 / 3 = 3.33
a // b	Divisão inteira	8 / 2 = 4
a % b	Resto	3 % 2 = 1

Operadores – Comparação

Os operadores de comparação são usados para comparar valores.

Operador	Descrição	Exemplo
a == b	a igual a b	3 == 3 # True
a != b	a diferente de b	5 != 2 # True
a < b	a menor que b	4 < 7 # True
a > b	a maior que b	10 > 20 # False
a <= b	a menor ou igual a b	8 <= 8 # True
a >= b	a maior ou igual a b	12 >= 10 # True

Operadores – Atribuição

Os operadores de atribuição servem para guardar valores em variáveis e também atualizar valores de forma mais rápida.

Devo usar sempre dessa forma?

"Não é obrigatório usar, mas é uma forma mais limpa e comum de escrever. Em projetos reais, isso melhora a leitura do código e evita erros."

Operador	Descrição	Exemplo
=	Atribui um valor à variável	x = 10
+=	Soma e atribui o resultado	x +=5
-=	Subtrai e atribui o resultado	x -=2
*=	Multiplica e atribui o resultado	x *=3
/=	Divide e atribui o resultado	x /= 2
//=	Divide (inteiro) e atribui	x //=2
%=	Calcula o resto e atribui	x %=3
**=	Eleva à potência e atribui	x **=2

Operadores – Associação

Os operadores de associação são usados para indicar um valor está dentro do outro.

Operador	Descrição	Exemplo
in	Está contido em	“a” in {“a” : 1}
not in	Não está contido em	“x” not in [1,2,3]

Operadores – Identidade

Os operadores de identidade são usados para verificar se duas variáveis apontam para o mesmo objeto.

Operador	Descrição	Exemplo
is	Mesmo objeto	a is b
is not	Objetos diferentes	a is not b

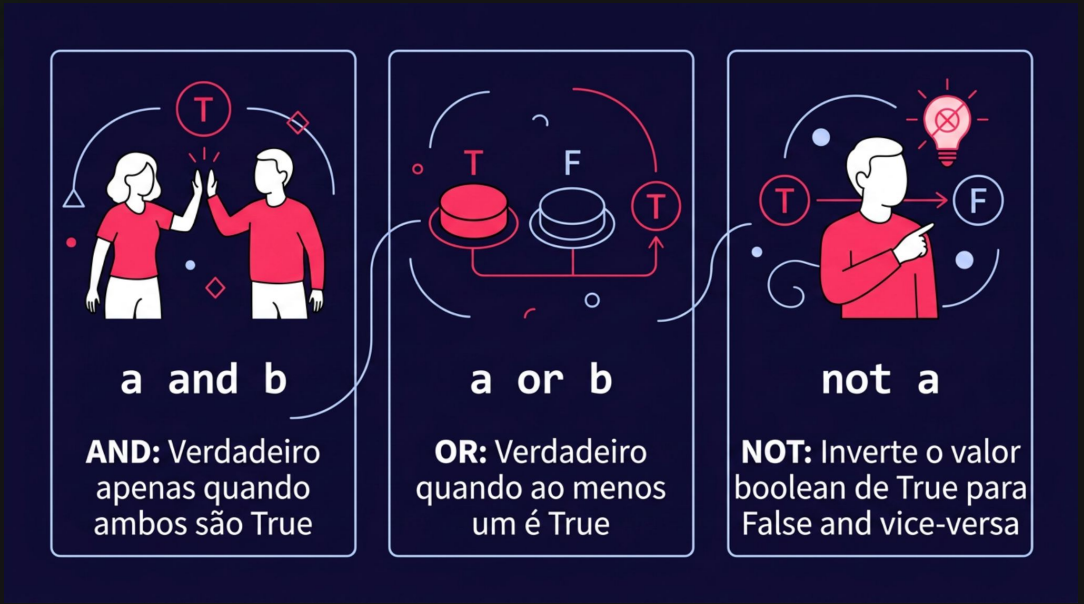
Operadores – Lógicos

Os operadores lógicos são usados para tomar decisões combinando condições.

Operador	Descrição	Exemplo
and	E (todas as condições precisam ser verdadeiras)	x > 5 and x < 10
or	OU (pelo menos uma condição precisa ser verdadeira)	x < 5 and x > 10
not	Nega a condição (inverte o resultado)	not(x > 5)

Lógica Booleana na Prática

A lógica booleana é o coração das decisões em programação. Todo sistema de um semáforo a um banco digital toma decisões baseadas em condições verdadeiras ou falsas. Entender as tabelas-verdade evita bugs e torna o código mais confiável.




```
# Exemplo prático: validação de acesso

usuario_ativo = True
senha_correta = True

pode_acessar = usuario_ativo and
senha_correta
print(pode_acessar)  # True

# Se a senha estiver errada:
senha_correta = False
pode_acessar = usuario_ativo and
senha_correta
print(pode_acessar)  # False
```

Questão de Entrevista Técnica

Vamos nos preparar?

"Explique a diferença entre

=,
==,
is

Quando você usaria cada um?"

Resposta

= Operador de Atribuição

Armazena um valor em uma variável. Não faz comparação.

Ex: nome = “Lorenzo" aqui “Lorenzo" é guardado na variável nome.

== Comparação de valor

Verifica se dois valores são iguais, retornando True ou False.

Ex: 5 == 5.0 retorna True, pois os valores são equivalentes.

Identidade do Objeto

Verifica se dois objetos são o mesmo objeto na memória.

Ex: a is None é o modo correto de checar se uma variável é nula em Python.

Uso = para atribuir, == para comparar valores e is para verificar se as variáveis pertencem ao mesmo objeto.

Expressões

Expressões

Uma expressão é qualquer combinação de valores, variáveis e operadores que o Python consegue avaliar e retornar um resultado

```
# Sem parênteses:
resultado = 2 + 3 * 4
# Python faz: 3 * 4 = 12, depois 2 + 12
print(resultado)  # → 14

# Com parênteses:
resultado = (2 + 3) * 4
# Python faz: 2 + 3 = 5, depois 5 * 4
print(resultado)  # → 20

# Outro exemplo de precedência:
x = 10 - 2 ** 3
# Potência primeiro: 2 ** 3 = 8
print(x)  # → 2

# Usando parênteses para controlar:
y = (10 - 2) ** 3
# Subtração primeiro: 10 - 2 = 8
print(y)  # → 512
```

💡 Regra de ouro: Use parênteses sempre que quiser garantir a ordem desejada. Eles tornam o código mais claro para quem lê!

A função print

A função print() é a principal forma de exibir informações ao usuário. Ela aceita múltiplos argumentos e possui parâmetros opcionais poderosos para controlar como os dados são exibidos.

```
print("Olá, mundo!")
print("Soma:", 3 + 5)
print("A", "B", "C")
# Saída: A B C

print("A", "B", sep="-")
# Saída: A-B
```

```
# Separador personalizado

print("Python", "é", "incrível", sep="-")
# → Python-é-incrível
```

1
sep =“”
Define o separador entre valores (padrão: espaço)

2
end=“”
Define o que aparece no final (padrão: nova linha)

✅ **Boas práticas: Use print() para entender o que está acontecendo no seu código durante o aprendizado. É sua melhor ferramenta de exploração!**

A Função input()

Com input(), seu programa pode **receber dados digitados pelo usuário**. Atenção: o valor retornado é sempre uma **string** , lembre-se de converter quando precisar de números!

```
# Recebendo o nome do usuário
nome = input("Qual é o seu nome? ")
print("Olá,", nome)

# Recebendo um número (precisa converter!)
idade = int(input("Quantos anos você tem? "))
print("Em 10 anos você terá", idade + 10, "anos.")

# Recebendo um número decimal
altura = float(input("Qual é a sua altura em metros? "))
print("Altura informada:", altura)
```

🔤 str (padrão)

Todo input() retorna texto. Sem conversão, não é possível fazer cálculos.

🔢 int(input())

Converte para número inteiro. Use para idades, contagens e quantidades.

💧 float(input())

Converte para número decimal. Use para altura, peso, preço e medidas.

Conversão de tipos

Como o próprio nome já diz, conversão de tipos é uma forma de converter um tipo x em outro tipo. Ex: número para string ou string para número.

Para fazer isso, o python disponibiliza três funções de conversão:

str(): o método str() do Python irá converter floats ou ints em string. Para usar essa função, passe um inteiro ou decimal.

int(): o método int() do Python irá converter floats ou strings, em inteiros. Para usar essa função, passe uma string ou um decimal.

float(): o método float() do Python irá converter inteiros ou strings em decimais. Para usar essa função, passe um inteiro ou string.

Float para string
str(1.1)
Converte um valor decimal em string

Inteiro para Float
preco_base = 50
preco_final = float(preco_base) # 50.0

Qualquer tipo para String
id_num = 12345
id_str = str(id_num) # "12345"

.format

Como o próprio nome já diz, conversão de tipos é uma forma de O .format() é uma outra forma de colocar variáveis em uma string.

Essa função vai ao final da string. Essa função necessita que para cada variável a ser adicionada na string, seja colocada uma máscara, representada pelas {}.

Cada chave representa uma variável. Cada variável é colocada entre os parênteses do .format e separado por vírgula, caso haja mais de uma variável.

```
# Por ordem posicional
print("Olá, meu nome é {} e sou {}.".format(“Jaime", "tutor"))

# Por índice (útil para repetir valores)
print("{0} estuda Azure, {0} estuda GCP.".format(“Jaime"))

# Por nome de argumento (chave-valor)
print("Projeto: {nome}. Camada: {camada}.".format(nome="Vendas", camada="Bronze"))
```

f-strings

As f strings são a forma mais simples e elegante de adicionar variáveis em uma string.
Para usá-las, basta colocar a letra f no início da string e, utilizar as chaves {} para colocar as variáveis, onde as variáveis serão colocadas dentro das chaves.

```
nome = “Ana"
cargo = "Data Engineer"
salario = 22000

# Exemplo básico
print(f"{nome} trabalha como {cargo}.")

# Operações matemáticas e métodos dentro da string
print(f"O bônus será de: {salario * 0.10}")
print(f"Nome em maiúsculo: {nome.upper()}")

# Formatação de casas decimais (muito útil em relatórios)
pi = 3.14159
print(f"Valor de pi: {pi:.2f}") # Resulta em 3.14
```

Boas práticas: f-strings

✅ Prefira f-strings ao invés de concatenação com [redacted] — mais legível

✅ Use `:.2f` para controlar casas decimais em números float

✅ Você pode chamar métodos dentro das chaves: `{nome.lower()}`

❌ Evite expressões muito complexas dentro das chaves porque pode prejudicar a leitura

PEP 8

Pep 8 – O guia de Estilo Python

O **PEP 8** é o guia oficial de boas práticas de formatação do Python. Seguir o PEP 8 torna o código mais legível, fácil de manter e profissional , seja trabalhando sozinho ou em equipe.

Indentação

Use **4 espaços** por nível. Nunca misture espaços e tabulações no mesmo arquivo.

Nomes de Variáveis

Use snake_case: nome_aluno, não nomeAluno nem NomeAluno.

Linhas em Branco

Separe blocos lógicos com uma linha em branco para melhorar a leitura.

Limite de Caracteres

Mantenha linhas com no máximo **79 caracteres**. Use \ ou parênteses para quebrar.

❌ Código sem PEP 8

```
x=int(input("Digite:"))
y=x*2+1
print("Resultado:",y)
```

✅ Código com PEP 8

```
valor = int(input("Digite um número: "))
resultado = valor * 2 + 1
print(f"Resultado: {resultado}")
```

Exercícios

Hora de Praticar!

Coloque em prática o que aprendeu sobre print(), input() e expressões. Tente resolver no Google Colab antes de ver a resposta!

1

Soma Simples

Peça ao usuário 2 números e exiba:

Soma

Dobro da soma

Produto entre os números

2

Apresentação Pessoal

Solicite nome, idade e cidade do usuário. Exiba uma mensagem formatada como: *"Olá! Meu nome é Michel, tenho 20 anos e moro em Gramado."*

3

Precedência em Ação

Calcule 5 + 3 * 2 e (5 + 3) * 2. Exiba ambos os resultados com print() e explique a diferença em um comentário no código.

💡 **Dica:** No Google Colab, execute cada célula com **Ctrl+Enter** e observe o resultado imediatamente!

Depuração – Encontrando e Corrigindo Erros

Depuração

Depurar (ou "debugar") é o processo de identificar e corrigir erros no código. Todo programador depura : inclusive os experientes! Conhecer os tipos de erro e as técnicas básicas economiza muito tempo.

1

SyntaxError

Erro de digitação na estrutura do código.
Exemplo: esquecer os dois-pontos : no final de um if. Python não consegue nem executar o arquivo.

2

RuntimeError / TypeError

Ocorre durante a execução. Exemplo: tentar somar uma str com um int sem converter primeiro: "5" + 3 gera TypeError.

3

Erro de Lógica

O código roda, mas dá resultado errado. Use print() em pontos estratégicos para "espiar" os valores intermediários e encontrar onde a lógica falha.

💡 **Dica:** Leia a mensagem de erro do Python de baixo para cima a última linha indica o tipo e local exato do erro. Não tenha medo das mensagens em vermelho!

Comentários

Depuração

Comentários são linhas no seu código que o Python simplesmente ignora. Eles servem como **documentação interna**, ajudando a explicar o que o seu código faz, como ele funciona, e por que certas decisões foram tomadas.

Comentários de Linha Única

Use o símbolo `#` para adicionar notas rápidas, seja em uma linha dedicada ou ao lado de uma instrução de código.

```
# Esta linha explica o propósito do bloco de código abaixo
usuario_ativo = True # Define o status do usuário como ativo
```

Explicando Lógica Complexa

Vá além do óbvio. Descreva o **porquê** de uma implementação, as suposições e as complexidades, não apenas o **o quê**.

```
# A validação abaixo garante que o CPF seja válido e único na base de
dados,
# seguindo o algoritmo de 11 dígitos e verificando a integridade do
formato.
# Isso evita fraudes e duplicação de cadastros.
if not validar_cpf(cpf) or not verificar_cpf_unico(cpf):
    raise ValueError("CPF inválido ou já cadastrado.")
```

Docstrings (Múltiplas Linhas)

Utilize três aspas duplas (`"""Docstring"""`) para documentar funções, classes e módulos. São vitais para a documentação formal e são acessíveis via `help()`.

```
def calcular_imc(peso_kg, altura_m):
    """
    Calcula o Índice de Massa Corporal (IMC) de uma pessoa.

    Args:
        peso_kg (float): O peso da pessoa em quilogramas.
        altura_m (float): A altura da pessoa em metros.

    Returns:
        float: O valor do IMC.
    """
    if altura_m <= 0:
        raise ValueError("Altura não pode ser zero ou negativa.")
    return peso_kg / (altura_m ** 2)
```

Google Colab

Google Colab

Plataforma na Nuvem

Ferramenta gratuita do Google para programação em Python diretamente no navegador, sem necessidade de instalação ou configuração local.

Notebooks Jupyter

Ambiente baseado em notebooks interativos que combinam código executável, texto formatado, imagens e gráficos em um único documento colaborativo.

Poder Computacional

Acesso a recursos de computação na nuvem como CPUs, GPUs e TPUs, ideais para projetos de ciência de dados, machine learning e inteligência artificial.

Integração Total

Conecta-se diretamente com Google Drive e GitHub, facilitando o salvamento, compartilhamento e colaboração em projetos de forma simples e eficiente.

Acesse a Plataforma

Entre em colab.research.google.com com sua conta Google

Montando o Drive e alguns atalhos

📂 Montando o Google Drive

Para acessar seus arquivos do Drive diretamente no Colab, execute numa célula de código:

```
from google.colab import drive
drive.mount('/content/drive')

# Agora leia arquivos normalmente:
with open('/content/drive/MyDrive/dados.txt') as f:
    print(f.read())
```

ℹ️ Na primeira vez, o Google pedirá permissão, clique em “Autorizar” e copie o código gerado.

🖼️ Inserindo Imagens em Markdown

```
![Descrição](https://url-da-imagem.png)

# Imagem do Drive (após montar):
from IPython.display import Image
Image('/content/drive/MyDrive/foto.png')
```

🔗 Inserindo Links em Markdown

Para anexar alguma URL, podemos fazer assim:

```
[Texto do link](https://site.com)

# Exemplo:
[Documentação Python](https://docs.python.org)
```

⌨️ Atalhos Essenciais do Colab

Ctrl+Enter
Executa a célula atual

Shift+Enter
Executa e avança para a próxima

Ctrl+M B
Insere célula de código abaixo

Ctrl+M M
Converte a célula para Markdown

Ctrl+M D
Exclui a célula atual

Estruturas

Aula 2

@SCTI.SC

Tuplas

Tuplas - Conceito

Tuplas são coleções ordenadas e imutáveis. Uma vez criadas, seus elementos não podem ser alterados. Isso as torna ideais para dados que não devem mudar.
Ex: coordenadas, configurações e retornos de funções.

```
# Concatenação (gera nova tupla)
t1 = (1, 2, 3)
t2 = (4, 5, 6)
t3 = t1 + t2
# (1, 2, 3, 4, 5, 6)

# Repetição
t4 = (0,) * 3  # (0, 0, 0)

# Verificar existência
print(3 in t1)  # True

# Contar ocorrências
dados = (1, 2, 2, 3, 2)
print(dados.count(2))  # 3

# Encontrar índice
print(dados.index(3))  # 3
```

Tuplas - Aplicações

Tuplas garantem que os dados **não serão alterados acidentalmente**. Isso é crítico em sistemas financeiros, coordenadas geográficas e registros de banco de dados.

Exemplos de uso

🔒 Segurança

Protege dados que não devem ser alterados acidentalmente

⚡ Performance

Tuplas são mais rápidas e consomem menos memória que listas

🔑 Chave de dicionário

Por serem hashbable, podem ser usadas como chaves de dicionários.

Tuplas - Aplicações

Mesmo sendo imutáveis, tuplas oferecem operações poderosas de acesso e manipulação que são amplamente utilizadas no mercado.

Aplicação Real: Sistema de Localização (Uber/Ifood)

```
locais = [
  ("São Paulo", -23.5505, -46.6333),
  ("Rio de Janeiro", -22.9068, -43.1729),
  ("Curitiba", -25.4284, -49.2733)
]

# Exibir todas as cidades
for cidade, lat, lon in locais:
    print(f"{cidade}: ({lat}, {lon})")

# Filtrar cidades no Sul/Sudeste
sul_sudeste = [c for c, lat, lon in locais if lat < -22]
print(sul_sudeste)
```

Aplicação Real: Produto e Preço

```
produtos = [
  ("Notebook", 3500.00, "eletronicos"),
  ("Mouse", 89.90, "eletronicos"),
  ("Cadeira", 1200.00, "moveis")
]
for nome, preco, categ in produtos:
    if preco > 500:
        print(f"{nome}: R${preco}")
```

Tuplas – Perguntas de Entrevistas

❓ Tupla vs Lista — qual usar?

Use tupla quando os dados são **fixos e imutáveis**. Use lista quando precisa modificar. Tuplas também são mais performáticas.

❓ Tupla pode ser chave de dict?

Sim! d = {(1,2): "ponto A"} é válido. Listas não podem ser chaves porque são mutáveis (não são *hashable*).

❓ Tupla Cite quando não devemos usar:

Os dados precisam ser atualizados

Você precisa adicionar/remover itens

A coleção é temporária e dinâmica

❓ Como iterar com índice?

Use enumerate(): for i, v in enumerate(tupla): — retorna índice e valor simultaneamente.

Listas

Listas - Conceito

Listas são coleções ordenadas e mutáveis que permitem armazenar múltiplos valores em uma única variável. Elas mantêm a ordem de inserção e permitem duplicatas.

Características Principais

- Ordenada: os elementos têm posição (índice)
- Mutável: pode adicionar, remover ou alterar elementos
- Permite duplicatas
- Suporta tipos mistos

Quando usar Listas?

- Quando a ordem importa
- Quando você precisa modificar os dados
- Filas de processamento, históricos, feeds

```
# Feed de Notícias (Estilo do X)


feed = ["Post A", "Post B", "Post C"]

# Novo post aparece no topo
feed.insert(0, "Post Novo")

# Exibir os 3 primeiros posts
for post in feed[:3]:
    print(post)

# Remover post antigo
feed.remove("Post C")

# Verificar tamanho do feed
print(f"Total de posts: {len(feed)}")
```

Listas - Aplicações

Grandes plataformas de streaming usam listas extensivamente para gerenciar filas de reprodução, histórico e recomendações.

Netflix – Fila de Reprodução

```
minha_lista = [
  "Stranger Things",
  "The Crown",
  "Dark"
]
# Adicionar novo título
minha_lista.append("Wednesday")
# Remover ao assistir
minha_lista.pop(0)
```

Spotify – Playlist Dinâmica

```
playlist = ["Música A",
            "Música B",
            "Música C"]
# Embaralhar simulado
# Remover música pulada
playlist.remove("Música B")
# Inserir na posição
playlist.insert(1,"Música D")
```

Listas – Operações Comuns

Adicionar e Remover

```
frutas = ["maçã", "banana"]
frutas.append("uva")      # adiciona no fim
frutas.insert(1, "kiwi")   # insere em posição
frutas.extend(["pera","melão"]) # adiciona vários itens
frutas.remove("banana")    # remove pelo valor
ultimo = frutas.pop() # remove e retorna último item
frutas.pop(0)   # remove por índice (posição 0)
```

Listas – Operações Comuns

Consultar e Ordenas

```
nums = [3, 1, 4, 1, 5, 9, 2]
print(nums.index(4))   # ( índice da primeira ocorrências de 4)
print(nums.count(1))   # 2 (quantidade de vezes que 1 aparece)
nums.sort()            # ordena crescente in-place
nums.sort(reverse=True) # ordem decrescente
nums.reverse()         # apenas inverte a ordem atual
copia = sorted(nums)   # cria uma nova lista ordenada crescente
```

Use sorted() para preservar a lista original. Use .sort() apenas quando a mutação da lista original for intencional e segura.

Listas - Slicing

O slicing permite extrair sublistas com a sintaxe lista[início:fim:passo]. O índice final é exclusivo. É uma das funcionalidades mais poderosas e elegantes do Python.

```
#Fatiamento em listas

nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

nums[2:5]     # [2, 3, 4]       → do índice 2 até 4
nums[:4]      # [0, 1, 2, 3]    → do início até 3
nums[6:]      # [6, 7, 8, 9]    → do índice 6 até o fim
nums[::2]     # [0, 2, 4, 6, 8] → de 2 em 2
nums[::-1]    # [9,8,7,...,0]   → invertida
nums[1:8:3]   # [1, 4, 7]       → início=1, fim=7, passo=3
```

Listas – Boas práticas de Mercado

- ✔ Prefira nomes no plural: `usuarios`, não `lista1`
- ✔ Use list comprehension no lugar de loops simples de criação
- ✔ Evite modificar listas enquanto itera sobre elas
- ✔ Use `tupla` se os dados não precisarem mudar (mais seguro e mais rápido)
- ✔ Prefira `enumerate()` a `range(len())`
- ✔ Documente o tipo esperado de elementos com type hints: `nomes: list[str]`

Listas – Perguntas de Entrevistas

❓ Pergunta 1

Qual a diferença entre o .append e o .extend()?

❓ Pergunta 2

Como inverter uma lista sem usar o .reverse

❓ Pergunta 3

Como remover duplicatas de uma lista mantendo a ordem?

❓ Pergunta 4

O que acontece ao fatiar além dos limites: lista[0:100]?

❓ Pergunta 5

Qual a complexidade de tempo do .insert() no início da lista?

❓ Pergunta 6

Como fazer uma cópia rasa vs. profunda de uma lista?

Dicionários

Dicionários – Conceito

Dicionários armazenam dados em pares **chave** → **valor**. São estruturas altamente otimizadas para buscas rápidas (O(1) em média) e são amplamente usados em APIs, configurações e manipulação de dados JSON.

```
# Perfil de usuário (estilo Facebook/LinkedIn)

usuario = {
  "id": 12345,
  "nome": "Ana Silva",
  "email": "ana@email.com",
  "plano": "premium",
  "ativo": True
}

# Acessar com segurança
nome = usuario.get("nome", "Anônimo")
plano = usuario.get("plano", "free")

# Verificar chave
if "email" in usuario:
    print(f"Email: {usuario['email']}")

# Atualizar dados
usuario["plano"] = "enterprise"
usuario["ultimo_login"] = "2024-01-15"
```

Métodos importantes

- .keys() — retorna todas as chaves
- .values() — retorna todos os valores
- .items() — retorna pares chave-valor
- .get(k, default) — acesso seguro
- .update(d) — mescla dicionários
- .pop(k) — remove e retorna valor

Dicionários – Aplicação no Mercado

E-commerce

```
carrinho = {
  "notebook": {
    "qtd": 1,
    "preco": 3500
  },
  "mouse": {
    "qtd": 2,
    "preco": 89.90
  }
}
total = 0
for item, dados in carrinho.items():
    total += dados["qtd"] *
dados["preco"]
print(f"Total: R${total:.2f}")
```

Cache de resultados

```
cache_cep = {}

ceps = ["01310-100",
        "20040-020",
        "01310-100"]

for cep in ceps:
    if cep not in cache_cep:
        # Simula busca na API
        cache_cep[cep] = f"Rua {cep}"
        print("Buscando API...")
    else:
        print("Cache hit!")
```

Agrupamento de dados

```
vendas = [
  ("SP", 1500), ("RJ", 900),
  ("SP", 2100), ("MG", 800),
  ("RJ", 1200)
]
por_estado = {}
for estado, valor in vendas:
    if estado not in por_estado:
        por_estado[estado] = 0
    por_estado[estado] += valor

# SP: 3600, RJ: 2100, MG: 800
```

Dicionários – Perguntas de Entrevistas

❓ Pergunta 1

O que é um Dicionário e qual a diferença para uma lista?

❓ Pergunta 2

Como acessar um valor em um Dicionário?

❓ Pergunta 3

Como adicionar uma nova chave em um Dicionário existente?

❓ Pergunta 4

Qual é a diferença entre .keys() , .values() e .items()?

❓ Pergunta 5

Como saber se uma chave existe em um Dicionário antes de acessá-la?

❓ Pergunta 6

Qual a diferença entre modificar um Dicionário com dicionario["chave"] = valor e usar .update()

Sets

Sets – Conceito

Sets são coleções não ordenadas, mutáveis e sem elementos duplicados. São extremamente eficientes para verificações de pertencimento e operações matemáticas de conjuntos como união, interseção e diferença.

```
# Criações e Operações

cores = {"vermelho", "azul", "verde"}
cores.add("amarelo")
cores.discard("roxo")  # não gera erro se
não existe

# Eliminar duplicatas de uma lista
numeros = [1, 2, 2, 3, 3, 4]
unicos = set(numeros)  # {1, 2, 3, 4}

# Operações de conjuntos
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)  # União:
{1,2,3,4,5,6}
print(a & b)  # Interseção:  {3, 4}
print(a - b)  # Diferença:   {1, 2}
print(a ^ b)  # Simétrica:   {1,2,5,6}
```

Sets – Onde aplicar?

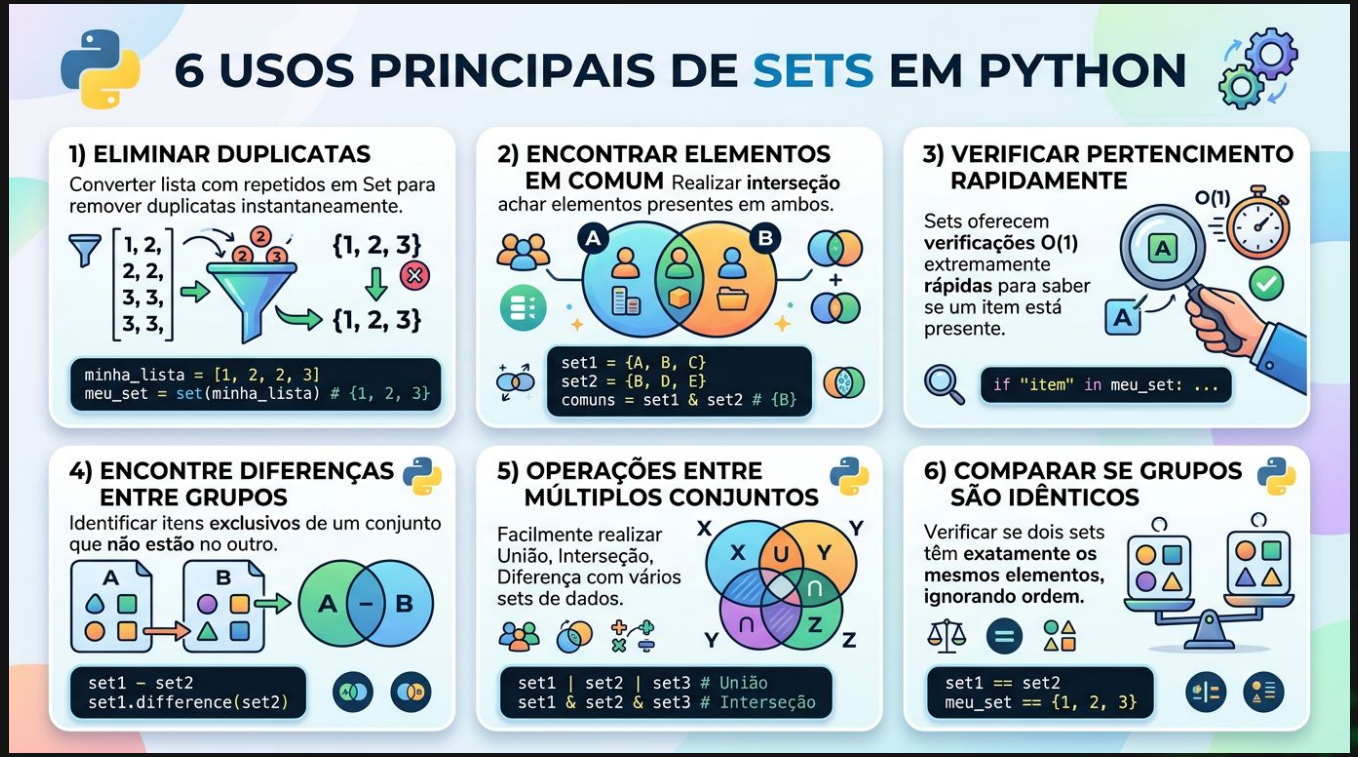
Encontrar elementos em comum
Identifique quais itens aparecem em múltiplos grupos simultaneamente.
Aplicação real: Usuários em múltiplas newsletters, clientes em diferentes categorias, permissões compartilhadas.

Verificar pertencimento rapidamente
Confirme instantaneamente se algo existe em uma coleção, sem importar o tamanho.
Aplicação real: Validar emails bloqueados, IPs autorizados, palavras censuradas, IDs de acesso.

Encontrar diferenças entre grupos
Descubra o que existe em um conjunto mas não no outro.
Aplicação real: Funcionários que saíram, itens faltantes em inventário, permissões necessárias vs. atuais.

Garantir sem duplicatas
Automaticamente descarta itens repetidos ao consolidar dados.
Aplicação real: Listas únicas de emails, IDs de clientes, catálogo de produtos de múltiplas fontes.

Sets – Onde aplicar?



Sets – Aplicação no Mercado

LinkedIn – Amigos em comum

```
conexoes_ana = {"Carlos", "Pedro",
                "Maria", "João"}
conexoes_bia = {"Maria", "João",
                "Lucas", "Pedro"}

em_comum = conexoes_ana & conexoes_bia
# {'Maria', 'João', 'Pedro'}

sugestao = conexoes_bia - conexoes_ana
# {'Lucas'} — sugerir para Ana
```

Filtrar Duplicatas

```
visitas = ["notebook", "mouse",
           "notebook", "teclado",
           "mouse", "notebook"]

# Remover duplicatas
produtos_vistos = set(visitas)
# {'notebook', 'mouse', 'teclado'}

# Quantos únicos?
print(len(produtos_vistos))  # 3
```

Controle de acesso

```
permissoes_usuario = {"ler",
                      "editar"}
permissoes_admin = {"ler", "editar",
                    "deletar",
                    "configurar"}

# Verificar permissão
acao = "deletar"
if acao in permissoes_usuario:
    print("Acesso permitido")
else:
    print("Acesso negado")
```

Escolhendo a Estrutura Correta

Saber quando usar cada estrutura é uma das habilidades mais valorizadas em entrevistas técnicas e no dia a dia de desenvolvimento.

Critério	Lista	Tupla	Dicionário	Set
Ordenado?	✅ Sim	✅ Sim	✅ (Python 3.7+)	❌ Não
Mutável?	✅ Sim	❌ Não	✅ Sim	✅ Sim
Duplicatas?	✅ Sim	✅ Sim	❌ (chaves únicas)	❌ Não
Acesso por índice?	✅ Sim	✅ Sim	Por chave	❌ Não
Caso de uso ideal	Filas, feeds	Coordenadas, registros	Perfis, configs	Tags, unicidade

Vamos nos preparar?

Vamos resolver alguns Exercícios?

Condicionais

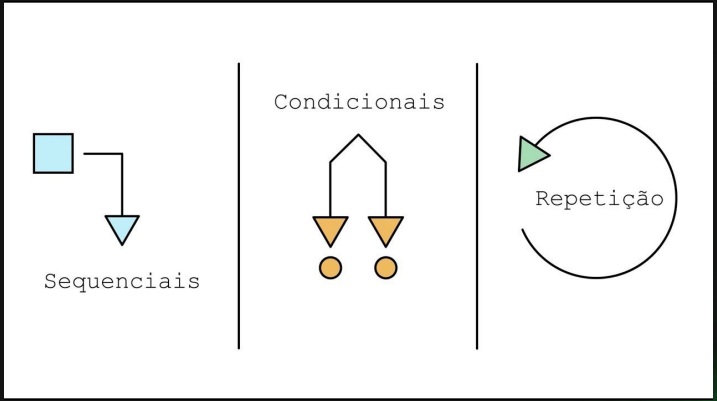
Aula 3

@SCTI.SC

O que são condicionais?

Em Python, as estruturas de controle de fluxo determinam quais instruções serão executadas e quantas vezes. Sem elas, um programa seria apenas uma lista linear de comandos — sem decisões, sem repetições, sem inteligência.

Imagine um programa como um mapa: as estruturas condicionais são as bifurcações e as de repetição são os atalhos que você percorre várias vezes até atingir seu destino.



Estruturas Condicionais - Introdução

IF

Verifica se uma condição é verdadeira e, se for, executa um bloco de código associado.

ELSE

Executa um bloco de código caso a condição associada ao if seja falsa.

ELIF

Abreviação de "else if", verifica uma segunda condição caso a condição associada ao if seja falsa e, se a condição do elif for verdadeira, executa um bloco de código associado.

Estruturas Condicionais – IF e IF/ELSE

IF é a estrutura condicional mais fundamental. Ele executa um bloco de código somente se a condição avaliada for True. O else define o que acontece quando a condição é False. A indentação (recuo de 4 espaços) é obrigatória em Python e ela delimita o bloco.

O bloco indentado só executa se a condição for verdadeira. A última linha executa sempre, pois está fora do bloco.

```
temperatura = 35

if temperatura > 30:
    print("Está muito quente!")
    print("Beba água.")

print("Programa encerrado.")
```

```
saldo = 150
valor_compra = 200

if saldo >= valor_compra:
    print("Compra aprovada!")
else:
    print("Saldo insuficiente.")
```

Estruturas Condicionais

As estruturas condicionais permitem que o programa tome decisões com base no valor de uma expressão booleana. É o mecanismo que transforma um script linear em um sistema capaz de reagir a situações diferentes e é essencial em qualquer aplicação real.

Sintaxe Fundamental

```
if condicao:
    # executa se True
elif outra_condicao:
    # executa se a anterior
    # for False e esta True
else:
    # executa se todas
    # as anteriores forem False
```

Exemplo: Classificação de Notas

```
if nota >= 9.0:
    print("Conceito A - Excelente!")
elif nota >= 7.0:
    print("Conceito B - Aprovado")
elif nota >= 5.0:
    print("Conceito C - Recuperação")
else:
    print("Conceito D - Reprovado")
```

! Atenção: Python usa **indentação** (4 espaços) para delimitar blocos. Erros de indentação são os mais comuns entre iniciantes.

Estruturas ELIF: Múltiplas Condições

O elif (contração de "else if") permite encadear múltiplas condições exclusivas. O Python avalia cada condição na ordem e executa apenas o bloco da primeira condição verdadeira, ignorando as demais. É mais eficiente e legível do que vários if separados.

Sintaxe Fundamental

```
nota = 75

if nota >= 90:
    conceito = "A - Excelente"
elif nota >= 80:
    conceito = "B - Bom"
elif nota >= 70:
    conceito = "C - Regular"
elif nota >= 60:
    conceito = "D - Insuficiente"
else:
    conceito = "F - Reprovado"

print(conceito)
```

Característica	Vários IF	IF + ELIF
Avalia todas as condições?	Sim, sempre	Para na 1ª verdadeira
Condições exclusivas?	Não	Sim
Performance	Menor	Maior
Legibilidade	Confusa	Clara

IF X ELIF: Qual a diferença?

Característica	Vários IF	IF + ELIF
Avalia todas as condições?	Sim, sempre	Para na 1ª verdadeira
Condições exclusivas?	Não	Sim
Performance	Menor	Maior
Legibilidade	Confusa	Clara

Boa prática: Use elif quando as condições forem mutuamente exclusivas (só uma pode ser verdadeira de cada vez). Use if separados quando múltiplas condições possam ser verdadeiras simultaneamente.

Vamos nos preparar?

Faça um programa que tenha duas variáveis numéricas e mostre na tela o maior deles.

Faça um Programa que peça um valor e mostre na tela se o valor é positivo ou negativo.

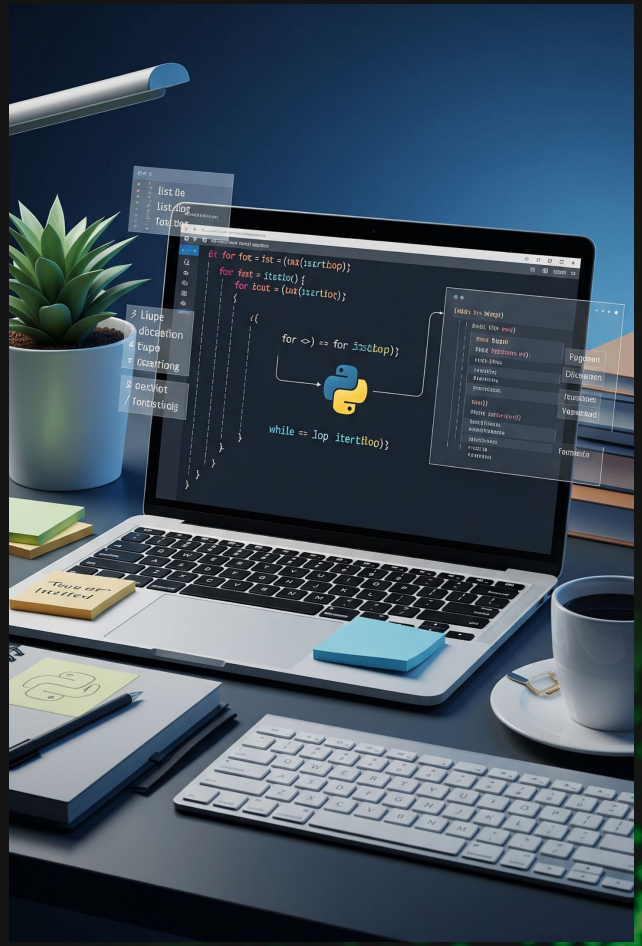
Crie um algoritmo que leia um número e alerte quando o mesmo for maior do que 100.

Faça um programa que pergunte o sexo de uma pessoa e se ela for masculino “M” ou feminino “F” ela pode passar, caso contrario informe que o sexo inserido foi invalido.

Condicionais de Repetição

Repetição é um dos pilares da programação.

Automação é umas tarefas essencial no mercado atual e depende fundamentalmente de loops. O Python oferece dois tipos: for para iterações com quantidade definida e while para repetições baseadas em condição



Loop: For

O for em Python é usado para iterar sobre qualquer sequência: listas, strings, tuplas, dicionários, e objetos gerados por range().

A cada iteração, a variável de controle assume o próximo valor da sequência automaticamente e sem necessidade de incremento manual.

```
for variavel in range(0, 0, 0):
```

Início
Fim
Passo

enumerate()

Retorna índice e valor simultaneamente.

```
for i, f in
enumerate(frutas):
    print(i, f)
```

range() com passo

Itera de 2 em 2, de 10 em 10, etc.

```
for n in range(0, 11, 2):
    print(n)  # 0,2,4,6,8,10
```

Palavras break e continue

break encerra o loop. continue pula para a próxima iteração.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Break

O break interrompe o loop completamente e passa para o código que vem depois. É útil quando você encontra o que está procurando ou quer parar por alguma razão.

```
for item in sequencia:
    if condicao:
        break  # Sai do loop
    # mais código
```

```
numeros = [10, 25, 30, 45, 50, 75, 100]
alvo = 45

for num in numeros:
    if num == alvo:
        print(f"Encontrado: {num}")
        break  # Para aqui
    else:
        print(f"Verificando: {num}")

print("Busca finalizada")
```

O que acontece:
Verifica 10 (não é 45)
Verifica 25 (não é 45)
Verifica 30 (não é 45)
Verifica 45 (encontrado!) → imprime e sai do loop
Nunca verifica 50, 75, 100
Imprime "Busca finalizada"
Aplicação real: Buscar um produto no estoque, procurar um cliente na lista, localizar um erro em dados.

Continue

O continue serve para pular a iteração atual do loop e ir direto para a próxima iteração, sem executar o resto do código naquele ciclo.

Casos de uso principais:
Filtrar dados inválidos — ignora registros que não atendem critérios
Pular condições específicas — salta números pares, usuários inativos, emails sem @
Evitar processamento desnecessário — não gasta recursos em dados que serão descartados mesmo
Manter o loop ativo — diferente de break, o loop continua rodando com os próximos itens.

```
emails = ["usuario@email.com", "invalido",
"outro@email.com"]

for email in emails:
    if "@" not in email:
        continue  # Pula emails inválidos

    print(f"Processando: {email}")
```

Na prática: Você tem uma lista de emails e quer processar apenas os que contêm "@". Os que não têm, você pula.

```
Processando: usuario@email.com
Processando: outro@email.com
```

Vamos nos preparar?

Faça um laço de repetição que printe os número de 1 até 20.

Faça um programa que print na tela a mensagem “olá mundo” 10 vezes.

Faça um laço de repetição que printe os números de 0 até 100, porém pulando de dois em dois.

Faça um laço de repetição que irá rodar 5 vezes e a cada volta ele terá que usar um input para pedir um número , mas somente print em tela os números maiores que 10

Loop: While

O while executa um bloco enquanto uma condição for verdadeira. É ideal quando não sabemos quantas iterações serão necessárias , por exemplo, validar a entrada do usuário até que ele forneça um valor correto, ou aguardar uma resposta de API.

```
while condição:
```

n1 < 100:

n1 != 100:

break()

Interrompe o loop imediatamente, saindo do bloco

continue

Pula para a próxima iteração, ignorando o restante do bloco atual.

else

Executando quando o loop termina normalmente (sem break)

Loop: While

O while repete um bloco de código enquanto uma condição for verdadeira. Diferente do for, ele não itera sobre uma sequência, ele continua executando até que algo mude a condição. Por isso, é essencial garantir que a condição se torne False em algum momento, evitando o temido loop infinito.

Ex: 1

```
contador = 1

while contador <= 5:
    print(f"Contagem: {contador}")
print("Loop encerrado.")
```

Ex: 2

```
senha = ""

while senha != "python123":
    senha = input("Digite a senha: ")

print("Acesso liberado!")
```

Ex: 3

```
tentativas = 0

while True:
    resposta = input("Adivinhe: ")
    tentativas += 1
    if resposta == "42":
        print(f"Acertou em {tentativas}x!")
        break
```

FOR x WHILE: Quando usar?

Situação	Use
Sabe quantas repetições	for
Itera sobre lista/sequência	for
Depende de condição externa	while
Aguarda input do usuário	while
Loop até evento ocorrer	while

Para fixar!

Estrutura	Tipo	Quando Usar	Exemplo Real
if	Condicional	Uma condição simples, verdadeira ou falsa	Verificar se usuário está logado
if/else	Condicional	Dois caminhos possíveis e exclusivos	Aprovar ou negar pagamento
if/elif/else	Condicional	Três ou mais condições exclusivas	Classificar nota, faixa etária, IMC
for	Repetição	Número de repetições conhecido ou iteração em coleção	Processar todos os itens de um carrinho
while	Repetição	Número de repetições desconhecido, depende de evento	Aguardar resposta válida do usuário

Casos de uso Reais no Mercado!

As estruturas de controle não existem apenas nos exercícios , elas são o coração de sistemas reais. Veja como cada estrutura aparece em aplicações profissionais do dia a dia.

E-commerce (IF/ELIF)

- Calcular frete por faixa de CEP
- Aplicar desconto por volume de compra
- Validar dados do cartão de crédito
- Classificar prioridade de pedidos

Análise de Dados (FOR)

- Processar cada linha de um CSV
- Calcular médias de uma lista de valores
- Gerar relatórios automáticos
- Enviar e-mails em lote para clientes

Sistemas (WHILE)

- Monitorar servidor até falha detectada
- Aguardar conexão de banco de dados
- Menus interativos de terminal
- Polling de APIs externas

Condicionais – Perguntas de Entrevistas

❓ Pergunta 1

Qual a diferença entre if e for?

❓ Pergunta 2

O que é um loop infinito e como evitá-lo

❓ Pergunta 3

O que é a estrutura while ?

❓ Pergunta 4

O que o break e continue fazem?

❓ Pergunta 5

Em quais situações devemos usar o for e o while?

❓ Pergunta 6

Defina: O que é range?

Vamos nos preparar?

Faça um laço while que mostre os números de 1 até 10

Faça um laço while que peça o nome de 5 pessoas e mostre esse nome em tela.

Faça um programa que peça 5 números para o usuário, porém só exiba o número se ele for par

Vamos nos preparar?

Escreva um programa que peça a senha do usuário se a senha for “PYTHON”, então você deve exibir uma mensagem de logado e encerrar o laço, caso contrário continue pedindo.

Faça um programa e peça para o usuário digitar um número secreto e se esse número for o 10 você deve exibir uma mensagem de parabéns e parar o laço, caso contrário peça para o usuário digitar outro número.

AVALIAÇÃO DOCENTE

O que você está achando das minhas aulas neste conteúdo?

Clique aqui ou escaneie o QRCode ao lado para avaliar minha aula.

Sinta-se à vontade para fornecer uma avaliação sempre que achar necessário.

SCTEC

PREPARANDO
VOCÊ PARA O
AGORA_

@SCTI.SC

SCTEC

PREPARANDO
VOCÊ PARA O
AGORA_

@SCTI.SC